

Wi-Fi Password Cracking — Demonstration Report

Project 20 · One offline tool, three attacks, on captured pcap files

Students: Mohammad Saifuzzaman (2105088)

Sayaad Muzahid Masfi (2105066)

Section / Group: B1 — Group 3

Course: CSE 406 — Computer Security

Scope and honesty statement. The tool reads captured pcap files and recovers WPA2-PSK passphrases from them; the attacks run on captured pcap files, and public sample captures from the Internet are used as input. Every number, transcript, and table in this report is real output produced by our own tool on the machines named; the commands to reproduce each one are listed in Section 9. Nothing in this report is estimated unless it is explicitly labelled “calculated,” and where a number is calculated rather than executed we say so and give the formula.

1 What we built

We wrote one tool, **wpa**crack (Python, standard library only — no aircrack-ng, no Hashcat, no scapy), that runs **three distinct password-cracking attacks** against captured WPA2-PSK handshakes, plus a defense analyzer. All three attacks share a single cryptographic core. The three attacks differ in *what they verify a guess against* and/or *where the candidate guesses come from*:

Attack	Candidate source	How a guess is confirmed
1. Handshake MIC	wordlist	derive the PTK, recompute the EAPOL Key MIC, compare
2. PMKID	wordlist	recompute $\text{PMKID} = \text{HMAC-SHA1}(\text{PMK}, \dots)$; needs only Message 1
3. Mask / brute	?d?l?u?s pattern	same two verifiers, candidates generated from a mask

The command-line surface is a single entry point with seven subcommands: **mic**, **pmkid**, **mask** (the three attacks); **analyze**, **policy**, **estimate** (defense); and **report** (a built-in self-test that re-runs every demo scenario and asserts each verdict).

2 System architecture

wpa	crack/crypto/	PMK (PBKDF2-HMAC-SHA1), PRF-512/PTK, MIC (KDV 1/2), PMKID — the shared verifiers
wpa	crack/pcap/	pcap + pcapng file reader (both byte orders)
wpa	crack/parse/	radiotap / 802.11 / EAPOL deframing, RSN IE + AKM, beacon SSID → HandshakeRecord
wpa	crack/attack_mic/	Attack 1 driver
wpa	crack/attack_pmkid/	Attack 2 driver
wpa	crack/attack_mask/	Attack 3 driver (mask expansion + time-box)
wpa	crack/defend/	analyze.py, policy.py, estimate.py
wpa	crack/sim/	synthetic handshake/pcap generator (builds the demo fixtures)
wpa	crack/cli.py	unified CLI: mic pmkid mask analyze policy estimate report
wpa	crack/tests/	16 automated tests (crypto known-answer + integration + real capture)

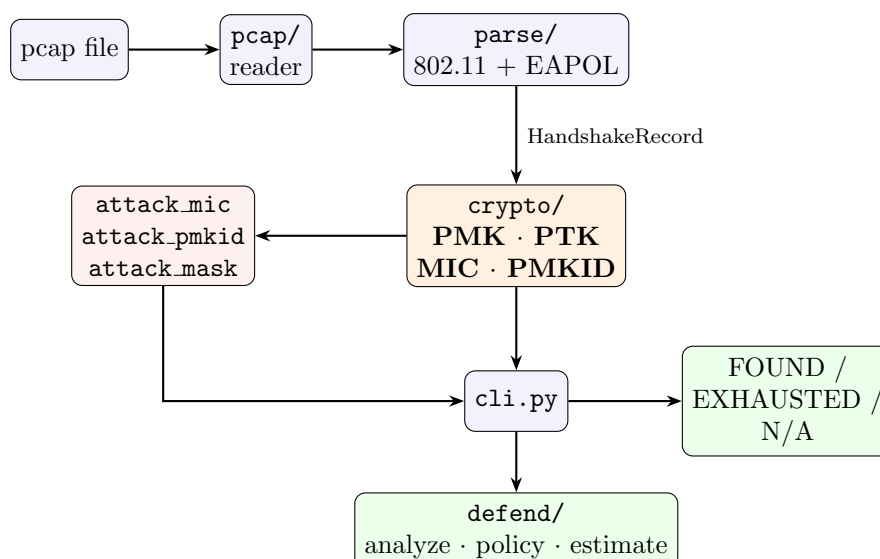


Figure 1: The pipeline. Only the attack module changes between the three attacks; the file reader, the 802.11/EAPOL parser, the crypto core, and the CLI are shared.

3 Protocol background and where we attack

WPA2-Personal never sends the passphrase or the derived key over the air. Instead, a station and access point run a **4-way handshake** that proves both sides know the Pairwise Master Key (PMK) and derives fresh session keys. Because the handshake carries values that are a function of the PMK, an attacker who *captures* the handshake can test passphrase guesses *offline*: for each guess, derive the PMK and check whether it reproduces a captured value.

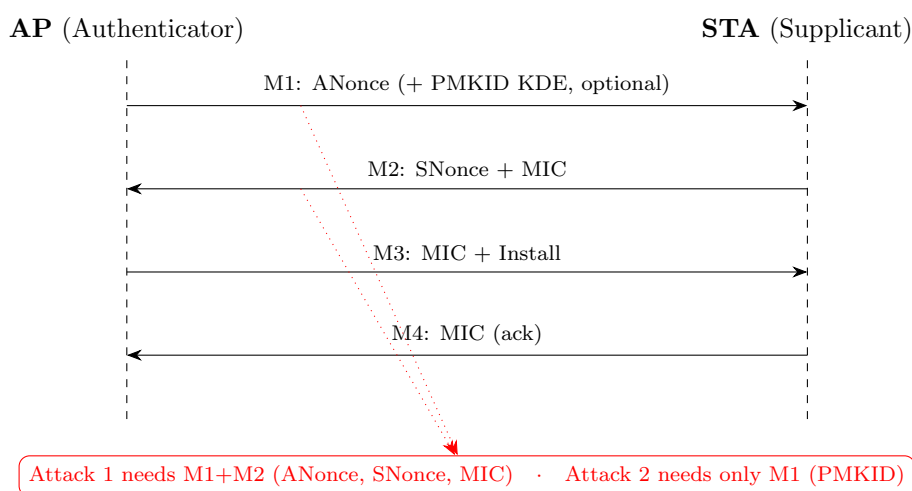


Figure 2: The 4-way handshake as seen in a capture, and which messages each attack needs. We recover the passphrase from passively captured frames only.

The verifiers our `crypto/` module implements (one implementation, shared by all three attacks)

are exactly:

$$\text{PMK} = \text{PBKDF2-HMAC-SHA1}(\text{passphrase}, \text{SSID}, 4096, 32), \quad \text{SSID is the raw salt}$$

$$\text{PTK} = \text{PRF-512}(\text{PMK}, \text{"Pairwise key expansion"}, \\ \min(AA, SPA) \parallel \max(AA, SPA) \parallel \min(An, Sn) \parallel \max(An, Sn))$$

$$\text{MIC} = \text{HMAC} \left(\underbrace{\text{PTK}[0:16]}_{\text{KCK}}, \text{EAPOL frame with MIC field zeroed} \right),$$

HMAC-MD5 or HMAC-SHA1 by Key Descriptor Version

$$\text{PMKID} = \text{HMAC-SHA1}(\text{PMK}, \text{"PMK Name"} \parallel AA \parallel SPA)[0:16]$$

The MIC algorithm is chosen by the Key Descriptor Version field (v1 = HMAC-MD5, v2 = HMAC-SHA1-128); v3 (AES-128-CMAC, 802.11w) is detected and reported as out of scope for this standard-library-only tool. All comparisons use `hmac.compare_digest`.

4 Demonstration

All commands below are run from the repository root. There are two kinds of input capture, and we are careful to distinguish them:

- **Synthetic fixtures** (`fixtures/*.pcap`) — built by our own `sim/` module with *known* passphrases, so every code path (each attack, the negative control, the WPA3 case) can be exercised deterministically.
- **Real, Internet-sourced captures** (`fixtures/public/`) — published sample captures from Wireshark and aircrack-ng, with documented passphrases. These prove the tool works on real 802.11 traffic, not just on captures it generated itself.

4.1 Built-in self-test on the synthetic fixtures

`python3 -m wpacrack report` regenerates the fixtures, runs all three attacks plus the negative control and the defense analyzer, and asserts each expected verdict. Real output:

```
=== attack results ===
[PASS] 1 MIC (dict hit)          -> FOUND pw='password123' (tried=3 164/s)
[PASS] 2 PMKID (dict hit)       -> FOUND pw='password123' (tried=3 180/s)
[PASS] 3 MASK (6-digit)        -> FOUND pw='001234' (tried=1235 166/s)
[PASS] 1 MIC (negative control) -> EXHAUSTED (tried=5 171/s)

=== defense analyzer ===
[OK-SAE] bssid=02:00:00:00:00:09 WPA3-SAE only: no offline PMK/PMKID target
[WARN-PSK] bssid=02:00:00:00:00:01 WPA2-PSK: offline dictionary / PMKID attack
possible
```

SELF-TEST: ALL PASS

4.2 Real, Internet-sourced captures

This is the strongest evidence that the tool is correct: it recovers the documented passphrase from four independent, publicly published captures it had no hand in creating. Real output:

```
$ wpacrack mic --pcap fixtures/public/wpa-Induction.pcap --wordlist wl --ssid
Coherer
```

```
[MIC] FOUND passphrase='Induction' (ssid=Coherer, tried=2,time=0.01s,rate
=174/s)

$ wpacli mic --pcap fixtures/public/wpa.cap --wordlist wl
[MIC] FOUND passphrase='biscotte' (ssid=test, tried=2,time=0.01s,rate
=174/s)

$ wpacli mic --pcap fixtures/public/wpa2.eapol.cap --wordlist wl --ssid
Harkonen
[MIC] FOUND passphrase='12345678' (ssid=Harkonen, tried=2,time=0.01s,rate
=172/s)

$ wpacli pmkid --pcap fixtures/public/test-pmkid.pcap --wordlist wl
[PMKID] FOUND passphrase='SP-91862D361' (ssid=WLAN-771698, tried=2,time=0.01s,rate
=174/s)
```

Capture (source)	SSID	Attack	Passphrase	Result
wpa-Induction.pcap (Wireshark)	Coherer	1 (MIC)	Induction	FOUND
wpa.cap (aircrack-ng)	test	1 (MIC)	biscotte	FOUND
wpa2.eapol.cap (aircrack-ng)	Harkonen	1 (MIC)	12345678	FOUND
test-pmkid.pcap (aircrack-ng)	WLAN-771698	2 (PMKID)	SP-91862D361	FOUND

The `test-pmkid.pcap` case is notable: it has *no* 4-way handshake at all, only a Message 1 carrying a PMKID. Attack 2 cracks it anyway, while Attack 1 correctly reports that it cannot run — exactly the honest behaviour we want:

```
$ wpacli mic --pcap fixtures/public/test-pmkid.pcap --wordlist wl
[MIC] N/A (ssid=WLAN-771698, no usable M1+M2 / M2+M3 handshake)
```

4.3 Was it successful, and why (or why not)?

Yes, in every case where the passphrase is reachable, and it correctly fails (without a false positive) where it is not:

- **Attack 1 (MIC)** succeeds whenever the capture contains a usable handshake pairing (M1+M2 or M2+M3) *and* the passphrase is in the wordlist — confirmed on three real captures above.
- **Attack 2 (PMKID)** succeeds whenever Message 1 carries a PMKID and the passphrase is in the wordlist — confirmed on a real capture with no full handshake.
- **Attack 3 (mask)** succeeds when the passphrase matches the mask pattern within the time-box — confirmed on the synthetic 6-digit fixture (001234).
- **It fails safely** when it should: a wrong wordlist yields `EXHAUSTED`, and a missing field yields `N/A`.

The reason it works is purely that WPA2-PSK exposes a PMK-derived value (the MIC or the PMKID) to anyone who captures the handshake, so guessing is an offline computation. The reason it *cannot* succeed against a strong passphrase is the size of the keyspace, quantified in Section 7.

5 Observed output on attacker, victim, and other hosts

Because this is an offline attack on a captured file, the “hosts” picture is different from a live attack:

- **Attacker host:** the console output shown throughout Section 4 — the parsed handshake fields (SSID, BSSID, station MAC), the number of candidates tried, the elapsed time and rate, and finally the recovered passphrase or a clean EXHAUSTED/N/A.
- **Victim / AP / other hosts:** *none, during the attack.* The only “victim-side” artifact is the captured `pcap` itself, which was produced before and independently of our tool (a published sample capture, or our own synthetic generator).

6 Performance: estimated rate vs. measured experiment

The design proposal estimated crack times from a single *assumed* candidate rate that was never measured. We closed that gap by actually measuring the rate — on CPU, and on GPU with our own PBKDF2-HMAC-SHA1 kernel written in PyTorch (RFC 2898 / IEEE 802.11i — again, not aircrack-ng or Hashcat). That kernel is a separate experimental artifact under `notebooks/` and is validated byte-for-byte against `hashlib.pbkdf2_hmac` before any timing is trusted; it does not change the stdlib-only tool. Measurements below are from a Kaggle instance with two Tesla T4 GPUs.

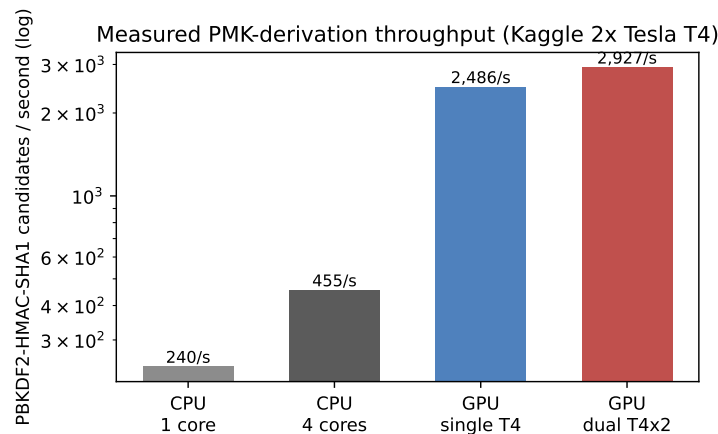


Figure 3: Measured PBKDF2-HMAC-SHA1 throughput: from 240 candidates/s on a single CPU core up to 2,927 candidates/s across two T4 GPUs — a 12.2× speedup on the same host. PBKDF2’s 4096 iterations per candidate are essentially the entire cost of every attack.

6.1 The GPU does not always win: a measured crossover

Our GPU kernel has a fixed per-call overhead, so at small candidate counts a plain CPU is actually faster; the GPU only wins above a break-even size. We measured this directly on a real capture (`wpa2.eapol.cap`) rather than assuming it:

6.2 Full-scale result on a realistic wordlist

The headline measurement is a genuine, realistic-scale run: SecLists’ full 5,189,454-entry common-password list against `wpa2.eapol.cap`, scanned to complete exhaustion (the real passphrase was excluded first so neither side could stop early).

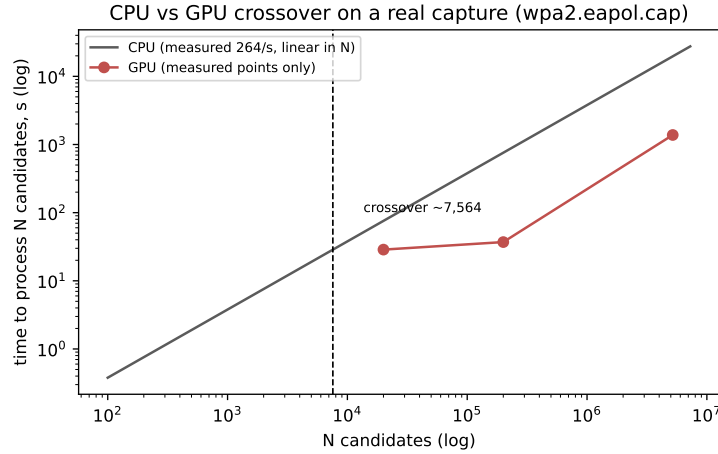


Figure 4: CPU time grows linearly with the wordlist size N (constant cost per candidate); the GPU’s three measured points do not. They cross at roughly 7,564 candidates: below that, plain CPU wins; above it, the GPU wins by an increasingly large margin.

Run	Candidates	Time	Basis
GPU, dual T4x2	5,189,454	23.0 min	measured (executed on Kaggle)
CPU, 1 core	5,189,454	5.5 h	measured (executed on Kaggle)
CPU, (MIC)	99,999	11.5 min	measured, independent 2nd machine (144/s)
CPU, (PMKID)	100,000	9.5 min	measured, independent 2nd machine (175/s)

The dual-GPU run of the full realistic list completed a real crack attempt in **23 minutes**; The two “independent 2nd machine” rows are a independently executed cross-check on a second computer, against two different real captures and both attack types — confirming the CPU rate is not a one-machine artifact. Below the $\sim 7,564$ -candidate crossover, however, plain CPU is the right tool: for the small dictionaries and short masks in our own demo (Section 4), CPU is not merely adequate, it is faster than the unoptimized GPU kernel.

7 Defense (countermeasure)

We built the countermeasure into the same tool, as three defensive subcommands.

1. Network analyzer (analyze). Reads the RSN AKM suite from a beacon and gives a verdict: WPA2-PSK $\Rightarrow$ **offline-crackable**; WPA3 transition mode $\Rightarrow$ **WPA2 fallback still crackable**; WPA3-SAE-only $\Rightarrow$ **no offline target**. On the real `wpa2.eapol.cap` beacon:

```
$ wpacrack analyze --pcap fixtures/public/wpa2.eapol.cap
```

```
[ANALYZE] WARN-PSK  bssid=00:14:6c:7e:40:80 akms=[2]
WPA2-PSK: offline dictionary / PMKID attack possible
```

2. Passphrase policy (policy). Flags short, low-diversity, or common-list passphrases. Real output for a weak and a strong passphrase:

```
$ wpacrack policy --passphrase "password123"
passphrase policy: WEAK (length 11)
- shorter than 15 characters (recommend >= 15 for WPA2-PSK)
- uses fewer than 3 character classes
- appears in a common-password list
```

```
$ wpacrack policy --passphrase "correct horse battery staple 9!"
passphrase policy: STRONG (length 31)
- meets length and character-class guidance; keep it unique and unguessable
```

3. Crack-time estimator (estimate). Uses our *measured* fastest rate (2,927 cand/s, dual T4) to show what actually survives an offline attack. This is real **estimate** output; the “expected” column is half the keyspace divided by the rate:

Passphrase shape	Keyspace	Expected crack time	Verdict
6 digits	10^6	2.8 minutes	WEAK
8 digits	10^8	4.7 hours	WEAK
8 lowercase letters	2.1×10^{11}	1.1 years	OK
8 full-ASCII chars	6.6×10^{15}	$\sim 35,900$ years	OK
12 full-ASCII chars	5.4×10^{23}	$\sim 3 \times 10^{12}$ years	OK
15 full-ASCII chars	4.6×10^{29}	$\sim 2.5 \times 10^{18}$ years	OK

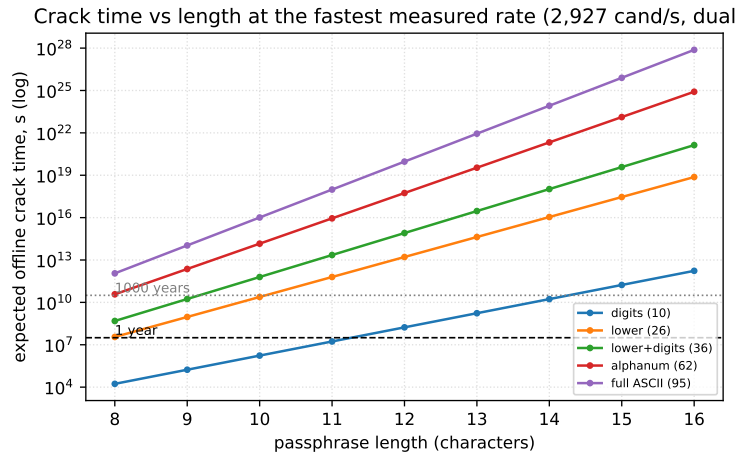


Figure 5: Expected offline crack time vs. passphrase length, per character set, at the fastest rate we measured (2,927 cand/s). Short or single-class passphrases fall below the one-year line; a long mixed passphrase is astronomically infeasible even with a GPU.

Recommendation. Use a long (≥ 15 character) mixed passphrase that is not in any wordlist; prefer WPA3-SAE-only and disable WPA2 transition mode (SAE / Dragonfly exposes no PMK-derived value to an offline capture); and keep SAE implementations patched against the Dragonblood class of attacks (Vanhoeft & Ronen, 2020). Crucially, the length recommendation holds *regardless* of whether the attacker uses a CPU or a GPU — the keyspace of a long passphrase dwarfs any realistic hardware speedup.

8 Correctness and honesty of the demonstration

- **Known-answer vector.** Our PMK matches the canonical IEEE 802.11i value for (password, IEEE): f42c6fc52df0ebef9ebb4b90b38a5f902e83fe1b135a70e23aed762e9710a12e.
- **Real captures.** The full PMK→PTK→MIC and PMK→PMKID paths recover the documented passphrases from four independent public captures (Section 4.2), proving the code matches real 802.11 and not merely our own generator.

- **Automated tests.** `pytest` runs **16 tests, all passing** (6 crypto known-answer tests, 9 integration tests, 1 real-capture test).
- **No false positives.** The negative-control scenario searches a wrong wordlist over a valid handshake and returns `EXHAUSTED`, never a bogus `FOUND`; all comparisons are constant-time full-digest checks.
- **Scope guard.** `tools/ethics_guard.py` fails the build if any socket, injection, or raw-transmit code appears anywhere in `wpacrack/`; it currently passes (*“ETHICS GUARD PASSED: wpacrack/ is file-in / verdict-out only.”*).

9 Reproducibility

Every result in this report is reproducible from the repository root:

```
python3 -m wpacrack.sim                # (re)build the synthetic fixtures

python3 -m wpacrack report --fixtures fixtures
                                     # the Section-4.1 self-test

bash tools/fetch_samples.sh           # download the public captures

python3 -m wpacrack mic --pcap fixtures/public/wpa-Induction.pcap \
    --wordlist WL --ssid Coherer

python3 -m wpacrack pmkid --pcap fixtures/public/test-pmkid.pcap \
    --wordlist WL

python3 -m wpacrack analyze --pcap fixtures/public/wpa2.eapol.cap

python3 -m wpacrack policy --passphrase "password123"

python3 -m wpacrack estimate --charset a --length 12 --rate 2927

python -m pytest -q                   # 16 tests (in a venv)

python3 tools/ethics_guard.py         # scope guard

python3 report/make_figures.py        # regenerate this report's figures
```

The GPU measurements (Section 6) are reproduced by the two notebooks under `notebooks/` on a Kaggle GPU instance; each begins with a validation cell that must pass before any timing is trusted.

Contributions: both group members (2105088, 2105066) contributed to the design, implementation, and testing; see the repository commit history for detailed authorship. References: IEEE Std 802.11i-2004; RFC 2898 / RFC 8018 (PBKDF2); He & Mitchell, NDSS 2005 (4-way handshake analysis); CERT-EU SA2018-019 (PMKID attack); RFC 7664 and Lancrenon & Škrobot, ISC 2015 (Dragonfly/SAE); Vanhoef & Ronen, IEEE S&P 2020 (Dragonblood).